

The S3 Closure Pool Against Its Own Definition

Recomputing every interaction score and every p-value from the singles, doubles and triples the 025 build joined, and the same on the deletion-only 029 build

Michael Volk

September 25, 2026

Abstract

The S3 closure pool holds, for every one of its 376,732 measured triples, every term of the equation that defines the triple's interaction score: the three singles, the three doubles and the triple itself, 1,121,645 records in all. Inside any one source screen that equation is exact: recomputing the digenic score from the fitness columns of the same row reproduces the reported score at $r = 0.999$ in Costanzo 2016 and $r = 1.000$ in Kuzmin 2018, and recomputing the trigenic score from the trigenic row together with the two single-mutant control query rows of the same screen, matched by array strain, returns the published value on 99.98 percent of the 91,111 Kuzmin 2018 triples to the rounding of its last digit. Inside the build it is not. The build's own fitness reproduces its own digenic scores at $r = 0.445$ and its own trigenic scores at $r = 0.230$, and the loss is traceable to the join rather than to the formula. Four mechanisms account for it. SGD essentiality enters as a fitness of 0 and is averaged with the measured fitness of the same gene's allele, which pulls 907 singles down by a median of 0.16 and shifts every score built on them by 0.12; taking the 0 back out halves the digenic error on the 139,461 doubles those singles touch. Screens are averaged across their own controls, so a single that Costanzo measured against 350 control screens is combined with the same gene's Kuzmin estimate and then used in an identity that was only ever true within one screen. Duplicate screens of the same genotype, 94 percent of the closure doubles, are merged into one record whose p-value is a t -test over the two or three duplicate scores with one or two degrees of freedom, which discards the source p-values entirely: the merged p correlates with the median source p at $\rho = 0.05$, and 1,754 doubles significant in every source are not significant in the build. And 99.6 percent of the Kuzmin 2020 digenic rows carry no query fitness, which the source scoring replaced by 1.0 before computing the score, a substitution that is recorded nowhere in the released table but is recovered here exactly. Where the build kept a single screen, its p-value is the source's and is reproducible from the double's own colony standard deviation at $\rho = 0.984$; the trigenic p-values are not reproducible from anything the build holds, because their null variance comes from control screens that were never released. The consequence for training on S3 is stated at the end: the fitness the model is asked to learn and the interaction it is asked to learn are not consistent with each other in this build, so the identity cannot be the mechanism by which fitness supervision helps, and every p-value in a merged record should be treated as unlabeled. A second build on the same graph, 029, keeps deletion alleles only and drops the merge, so the value a record carries becomes a label policy chosen at read time; under four such policies the identity reproduces each screen's own scores and no other's, Costanzo's digenic scores at $r = 0.90$ and 0.75 under a Costanzo-first policy and Kuzmin's 2018 trigenic scores at $r = 0.52$ under a Kuzmin-first one, while the policy that is best for one order is worst for the other. Which screen's standard a model learns is a decision the query and the policy make, and the cost of each choice is stated per screen. What separates that from the within-screen control is a single term, the double-mutant query strain's fitness, which every trigenic row reports and no record in either build holds: running the published asymmetric form directly against the symmetric one on identical records moves the 2018 screen by 0.007, so the shape of the equation is excluded and the missing measurement accounts for the gap.

Section status:  ready  author-reviewed, keep stable  not done, agents may edit freely

Contents

1	The question 	2
2	How the pool was built 	2
3	Strength: the score against its own definition 	4
4	Confidence: the p-value against what the build holds 	7
5	The hazards of the join, counted 	8
6	What this means for training on S3 	9
7	The same identity on a deletion-only query 	9
8	The multi-measurement build 	13
9	Limits 	14

1 The question x

A genetic interaction score is not a measurement. It is a difference between a measured fitness and a fitness expected from lower-order measurements, and the expectation is a product. For two deletions a and b with single-mutant fitness f_a and f_b and double-mutant fitness f_{ab} ,

$$\varepsilon_{ab} = f_{ab} - f_a f_b, \quad (1)$$

and for three, with the three doubles already scored,

$$\tau_{abc} = f_{abc} - f_a f_b f_c - \varepsilon_{ab} f_c - \varepsilon_{ac} f_b - \varepsilon_{bc} f_a. \quad (2)$$

Both equations are stated verbatim in the supplementary text of Kuzmin 2018 and Kuzmin 2020 (mirror keys `kuzminSystematicAnalysisComplex2018`, `kuzminExploringWholegenomeDuplicate2020`, `si/si1.md` lines 84 to 91 and 64 to 67), and Eq. 1 in Costanzo 2016 and Baryshnikova 2010. Substituting Eq. 1 into Eq. 2 gives the form used here, $\tau_{abc} = f_{abc} - f_a f_b f_c - f_{ab} f_c - f_{ac} f_b - f_{bc} f_a + 2f_a f_b f_c$.

The S3 closure pool of the 025 build was assembled so that a model trained on it sees every term of Eq. 2 for every triple it is asked to score: all 5,694 single-deletion records in the build, the 739,219 doubles whose gene pair lies inside some measured triple, and all 376,732 triples. The training arm (`cgt_s3_r_kl_fit_031`, three seeds on IGB `mmli`) supervises fitness and interaction jointly, with the 010 random split pinned over the triples. That design carries an assumption worth testing before the runs finish: that the fitness values in the pool and the interaction values in the pool are consistent with each other under the equations that define the latter.

Two questions follow, and the document answers both against the source tables the build was assembled from.

Strength. Does the fitness stored in the build reproduce the interaction stored in the build? Eq. 1 on every closure double and Eq. 2 on every triple, the recomputed score against the stored one, and the same recomputation on the raw source rows as a control for the formula.

Confidence. Does anything stored in the build reproduce the p-value stored in the build? Each interaction record carries a p-value from its source screen, or, when the build merged several screens of the same genotype, a p-value the build computed itself. The recomputation uses the stored standard deviations under the normal model the sources describe, and for merged records compares the stored p against the p-values the sources reported for the same genotype.

Three earlier readings frame the answer. The recapitulation of 2026-09-04 (`recapitulate_tmi_from_fitness.py`) put the trigenic identity on the whole build at $r = 0.23$ and concluded that noise accumulation, not a sign or scale error, was the cause. The 025 loaders' provenance work established that the released combined-mutant standard deviation is a colony sample SD over four colonies and that a two-sided normal test on it ranks the reported p-values at $\rho = 0.985$. And the additive-baselines report established that a per-gene additive ridge reaches $r = 0.40$ on these labels without any fitness at all. What is new here is the decomposition: which step of the join costs what, measured on the pool a model is about to be trained on.

2 How the pool was built x

Figure 1 traces a record from its source screen to the trainer. Each step is a committed piece of code, so each hazard has a line number rather than a suspicion.

The sources. Costanzo 2016 contributes single-mutant fitness at 26 and 30 degrees, double-mutant fitness with a colony standard deviation, and a digenic score with a p-value, one row per query-array pair per screen, 12.2 million double rows over four released files. Its single-mutant fitness is not screen-specific: the supplementary text (`costanzoGlobalGeneticInteraction2016/si/si1.md` line 94) states that array strains were “based on an average of 350 replicate control screens” and query strains on 17, so f_a and f_b in Eq. 1 are genome-wide standards. Kuzmin 2018 and Kuzmin 2020 contribute single, double and triple fitness, digenic and trigenic scores and their p-values, one row per query-array pair, 501 thousand and 1.4 million rows. Their array single-mutant fitness is taken from Costanzo 2016 (`kuzminSystematicAnalysisComplex2018/si/si1.md` line 135: “single mutant fitness estimates available from a previous study”), and the two digenic terms of Eq. 2 come from the single-mutant control query screened “in the same batch” as the double-mutant query (line 75). SGD essentiality contributes a label with no fitness measurement, and SynthLethDB a synthetic-lethal pair with none.

The build. `CompositeFitnessConverter` turns an essentiality or synthetic-lethality entry into a fitness of 0. `MeanExperimentDeduplicator` (`torchcell/data/mean_experiment_deduplicate.py`) then merges every entry that shares an experiment type and a sorted gene set: fitness becomes the mean and its standard deviation the root mean square (lines 308 to 316); an interaction score becomes the mean, and its p-value becomes a two-sided t -test of the

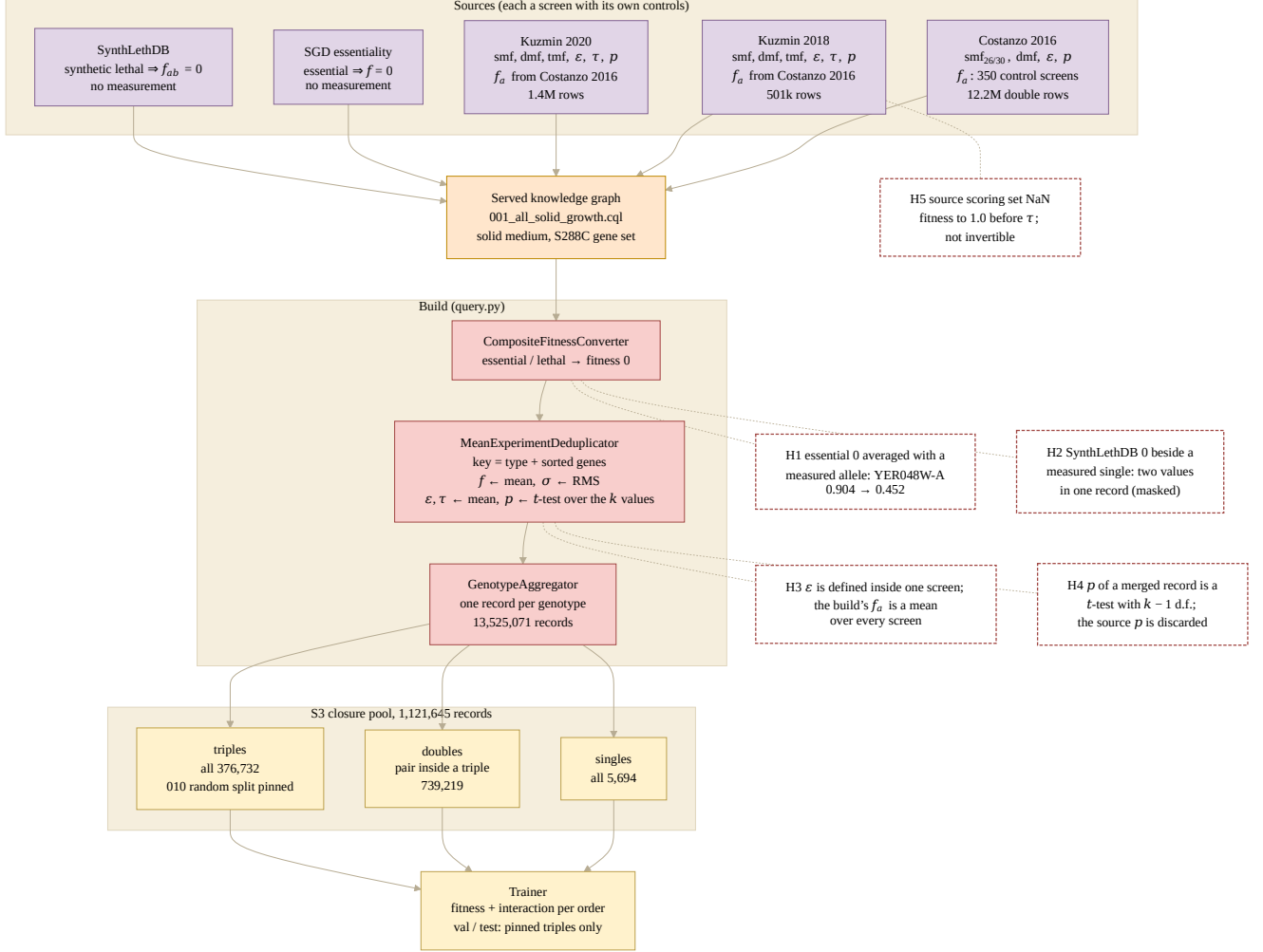


Figure 1: From five sources to one training pool. Purple: the sources, each a screen with its own controls and its own single-mutant fitness standard. Orange: the Cypher query over the served knowledge graph (`experiments/025-solid-growth/queries/001_all_solid_growth.cql`), filtered to solid medium and the S288C gene set. Red: the build (`experiments/025-solid-growth/scripts/query.py`): essentiality and synthetic lethality become a fitness of 0, duplicates of one experiment type and one gene set are merged into a mean, and records are grouped per genotype. Yellow: the S3 pool and the trainer. Dashed boxes H1 to H5 are the hazards quantified in Sections 3 to 5; each attaches to the step that introduces it.

merged scores against zero with $k - 1$ degrees of freedom (lines 480 to 494), which does not read the source p-values at all. **GenotypeAggregator** groups what remains into one record per genotype, 13,525,071 of them. The trainer reads one fitness value and one interaction value per record; a record that still carries two fitness values, which happens when a SynthLethDB entry lists the same gene twice and so escapes the merge key, is masked (three records, Section 5).

The pool. Table 1 lists what the join produced, by perturbation order and by the set of sources behind each record’s fitness. Of the 5,694 singles, 1,140 carry an SGD essentiality entry, and 233 of those carry nothing else. Of the 739,219 closure doubles, 697,825 (94.4 percent) are merged from two or more screens, most often the two orientations and two temperatures at which Costanzo screened the same pair. Of the 376,732 triples, 12,914 are merged, mostly the same triple reached from two different double-mutant queries.

Table 1: Records of the S3 pool by perturbation order and by the sources joined into their fitness value. Generated by `s3_closure_recompute.py` (`results/s3_closure_composition.csv`). “GeneEssentialitySgd” means an essentiality entry converted to a fitness of 0 entered the mean.

order	fitness sources joined	records
1	SmfCostanzo2016	3,193
	SmfCostanzo2016 + SmfKuzmin2018	1,111
	GeneEssentialitySgd + SmfCostanzo2016	673
	GeneEssentialitySgd + SmfCostanzo2016 + SmfKuzmin2018	234
	GeneEssentialitySgd	233
	SmfCostanzo2016 + SmfKuzmin2020	185
	SmfCostanzo2016 + SmfKuzmin2018 + SmfKuzmin2020	41
	SmfKuzmin2018	12
	SmfKuzmin2020	9
	SmfCostanzo2016 + SynthLethalityYeastSynthLethDb	2
2	SmfCostanzo2016 + SmfKuzmin2018 + SynthLethalityYeastSynthLethDb	1
	DmfCostanzo2016 + DmfKuzmin2020	514,325
	DmfCostanzo2016 + DmfKuzmin2018	166,377
	DmfKuzmin2020	37,148
	DmfCostanzo2016 + DmfKuzmin2018 + DmfKuzmin2020	14,791
	DmfKuzmin2018	5,358
	DmfCostanzo2016 + DmfKuzmin2020 + SynthLethalityYeastSynthLethDb	351
	DmfKuzmin2018 + DmfKuzmin2020	321
	DmfCostanzo2016 + DmfKuzmin2018 + SynthLethalityYeastSynthLethDb	277
	DmfCostanzo2016	208
	DmfCostanzo2016 + DmfKuzmin2018 + DmfKuzmin2020 + SynthLethalityYeastSynthLethDb	31
	DmfCostanzo2016 + SynthLethalityYeastSynthLethDb	17
	DmfKuzmin2020 + SynthLethalityYeastSynthLethDb	8
	DmfKuzmin2018 + SynthLethalityYeastSynthLethDb	5
	SynthLethalityYeastSynthLethDb	2
3	TmfKuzmin2020	285,682
	TmfKuzmin2018	90,567
	TmfKuzmin2018 + TmfKuzmin2020	483

3 Strength: the score against its own definition ✗

The formula is exact inside a screen. Figure 2a and the first three rows of Table 2 are the control. On the raw Costanzo 2016 rows the identity reproduces the reported score at $r = 0.999$, slope 1.00, root-mean-square error 0.002; on the raw Kuzmin 2018 rows at $r = 1.000$ with zero residual; on the raw Kuzmin 2020 rows at $r = 0.996$. The Kuzmin 2020 figure needs one substitution to reach that: 99.6 percent of its digenic rows carry no query single-mutant fitness, and the identity reproduces the reported score only when that missing value is set to 1.0, which is what the supplementary text says the scoring did (“all NaN fitness estimates were assigned a value of 1.0”, `si/si1.md` line 41). Hazard H5 is therefore not a hypothesis: the reported Kuzmin 2020 digenic scores were computed against a query fitness of 1.0, and the build inherits scores that no stored fitness can reproduce for those pairs. The small vertical streak at a reported score of 0 in Figure 2a is the 2.1 percent of Costanzo rows with a missing query fitness, where the same rule applies.

Digenic, in the build. The same identity on the build’s own values reaches $r = 0.445$, $\rho = 0.493$, slope 0.99 and a root-mean-square error of 0.072 over 728,856 doubles (Figure 2b). The slope says the identity holds in expectation; the error says it fails in value, by an amount larger than the stored score’s own spread. The strata separate the causes. Doubles whose two singles carry no essentiality entry reach $r = 0.598$ with error 0.042; doubles touching an essentiality-tainted single reach $r = 0.452$ with error 0.140 and an intercept of 0.116, which is the signature of a product $f_a f_b$ that is too small because one factor was averaged with a 0. Restoring the measured mean of the tainted single moves that

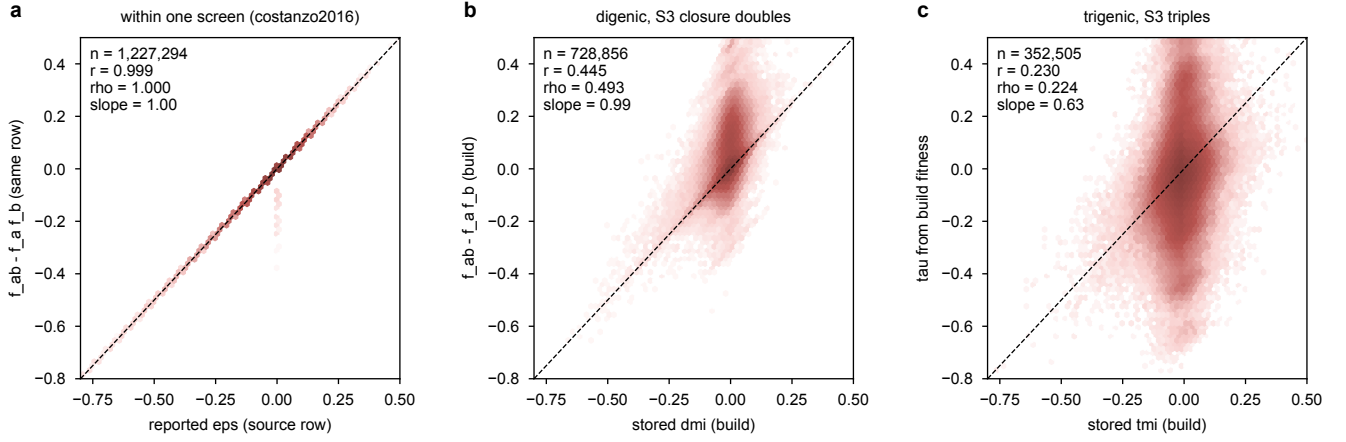


Figure 2: Recomputed against stored interaction scores. (a) Within one source screen: each Costanzo 2016 row’s $f_{ab} - f_a f_b$ against the score reported on the same row, over the 1,227,294 rows whose gene pair is a closure pair. (b) In the build: Eq. 1 from the build’s single and double fitness against the stored digenic score, all 728,856 closure doubles with both. (c) In the build: Eq. 2 from the build’s fitness against the stored trigenic score, the 352,505 triples with full closure. Hexagon shade is log count; the dashed line is identity.

Table 2: Recomputed against stored score, by stratum. Within-screen rows recompute from the columns of one raw source row; every other row recomputes from the build’s stored fitness. “Measured mean restored” takes the essentiality 0 back out of each tainted single by $k/(k-1)$ (Section 5). The best r is bold. Generated by `s3_closure_recompute.py`.

stratum	n	r	ρ	slope	rmse
within one screen, Costanzo 2016 (raw row)	1,227,294	0.999	1.000	1.00	0.002
within one screen, Kuzmin 2018 (raw row)	197,876	1.000	1.000	1.00	0.000
within one screen, Kuzmin 2020 (raw row)	601,330	0.996	0.997	1.00	0.004
digenic, all closure doubles	728,856	0.445	0.493	0.99	0.072
digenic, one screen only	36,499	0.339	0.314	0.95	0.120
digenic, merged screens	692,357	0.459	0.509	0.99	0.068
digenic, no essential single	589,395	0.598	0.633	0.98	0.042
digenic, an essential single (stored mean)	139,461	0.452	0.382	0.97	0.140
digenic, an essential single (measured mean restored)	139,461	0.491	0.441	0.96	0.071
trigenic, all triples	352,505	0.230	0.224	0.63	0.171
trigenic, one screen only	339,616	0.239	0.233	0.66	0.169
trigenic, merged screens	12,889	0.114	0.153	0.30	0.223
trigenic, no essential single	278,060	0.205	0.211	0.58	0.169
trigenic, an essential single (stored mean)	74,445	0.255	0.188	0.60	0.180
trigenic, an essential single (measured mean restored)	74,445	0.250	0.186	0.60	0.167
trigenic, from stored dmi	350,318	0.213	0.174	0.48	0.144

stratum to $r = 0.491$, intercept 0.014, error 0.071: half the error of that stratum was the essentiality 0. Doubles kept from one screen reach only $r = 0.339$, against $r = 0.459$ for merged doubles, because a merged double’s fitness and score are both means over screens and average some of the same noise away, while a single-screen double’s fitness is compared against singles averaged over other screens.

Trigenic, in the build. Eq. 2 on the build’s fitness reaches $r = 0.230$, slope 0.63 and error 0.171 over the 352,505 triples whose three singles and three doubles are all present (Figure 2c). The error is 2.7 times the standard deviation of the stored trigenic score, so the reconstruction carries more propagated measurement error than signal, which is the attenuation the 2026-09-04 recapitulation computed. The essentiality strata behave as in the digenic case (intercept -0.085 with the stored singles, 0.022 with the measured mean restored) without moving r , because five terms of noise remain. Triples merged from two queries reach $r = 0.114$, and using the stored digenic scores in place of the recomputed products gives $r = 0.213$, no rescue. At the intermediate threshold of $|\tau| > 0.08$ the build’s fitness calls 169,476 triples where the stored scores call 46,889, and the two agree on 29,327, a recall of 0.63 and a precision of 0.17.

The trigenic formula is exact inside a screen, and one term carries it. The digenic control above fits on one raw row. The trigenic identity does not, because the supplementary methods place its terms in three different measurements (`kuzminSystematicAnalysisComplex2018/si/si1.md` line 191): the double-mutant query fitness f_{ij} , the array single fitness f_k and the triple fitness f_{ijk} sit on the trigenic row itself, while ε_{ik} , ε_{jk} and the query singles f_i , f_j come from the two single-mutant control query strains, gene plus the *HO* deletion, screened against the same array strain in the same batch. Matching those rows by array strain identifier recovers the released score exactly, and the form that does it names what the source used:

$$\tau_{ijk} = f_{ijk} - f_{ij}f_k - \varepsilon_{ik} - \varepsilon_{jk},$$

that is, Eq. 2 with the single-mutant query fitness set to 1. Pairing each trigenic row to its two control query strains through the released query-strain list rather than by gene name, which matters for the five 2018 genes that carry two control strains, this reproduces the published adjusted score on 99.98 percent of the 91,111 Kuzmin 2018 rows and 99.56 percent of the 301,798 Kuzmin 2020 rows to within 10^{-4} , at a median absolute residual of 2.7×10^{-5} , which is the rounding floor of the four-decimal fitness columns rather than model error (Table 3). Matching the control screens by gene name instead, and averaging where a gene has two, costs 5 percent of the 2018 rows and 99 percent of those failures fall on those five genes. The model in Eq. 2 is therefore the right one and is implemented correctly here; nothing in the published correction is missing.

Written on the released columns the same statement is an exact identity rather than a reconstruction. The trigenic row’s own raw score is $\varepsilon_{ij,k} = f_{ijk} - f_{ij}f_k$, so

$$\tau_{ijk} = \varepsilon_{ij,k} - \varepsilon_{ik} - \varepsilon_{jk},$$

the row’s raw score minus the two control queries’ adjusted scores at the same array strain, which holds to a median residual of 6.5×10^{-19} on Kuzmin 2018 and exactly zero on Kuzmin 2020. Two routes therefore reconstruct the published score, one through the query strain’s fitness and one through the row’s raw score, and the build holds neither column.

The substitution of 1 for the query singles is a property of the released values, and it is not what the supplementary methods describe. Both papers write the model with the two control scores weighted by f_i and f_j , and the 2018 text says explicitly that these “are single mutant fitness estimates available from a previous study” (`si/si1.md` line 135). Neither says anywhere that the weights are 1. For Kuzmin 2020 the two are reconcilable, since only 0.4 percent of its control rows carry a query fitness and its methods state that every missing fitness was assigned 1.0 during scoring. For Kuzmin 2018 they are not: 99.2 percent of its control rows carry the measurement, the released fitness standard carries it for 165 of the 182 query strains, and using either reproduces 4.3 and 5.5 percent of rows against 99.98 percent for unit weights. So the released 2018 scores were computed with weights the methods do not describe. What a control screen contributes to a trigenic score, in the values as published, is its interaction and not its fitness.

Table 3: The trigenic identity inside one screen, matched by array strain identifier. The first row of each pair takes every term from the screen that measured it. The second drops the double-mutant query fitness f_{ij} , which is the only change made, and is what a closure over the database is forced into because that measurement is not a record. Generated by `s3_closure_trigenic_within_screen.py`.

screen	terms	n	r	ρ	slope	rmse
Kuzmin 2018	every term from the same screen	91,111	0.990	0.982	0.99	0.008
	the query double fitness dropped	91,111	0.495	0.422	1.05	0.138
Kuzmin 2020	every term from the same screen	301,706	0.976	0.975	0.97	0.014
	the query double fitness dropped	301,706	0.420	0.491	0.97	0.151

That leaves one term to explain the build. Dropping f_{ij} alone, and changing nothing else, collapses the same reconstruction to $r = 0.499$ on Kuzmin 2018 and $r = 0.479$ on Kuzmin 2020. The double-mutant query fitness is the term

that carries the trigenic score, which is the same statement as saying that the digenic interaction between the two query genes is large and cannot be inferred from their single-mutant fitness. The build has no record of it: across the whole Kuzmin 2018 screen only 172 query pairs carry a released double-mutant query fitness, 129 of them appear as a double in the 029 build, and those records are a different measurement of the same genotype, an array screen by Costanzo or by Kuzmin’s own digenic panel, agreeing with the query-strain value at $r = 0.777$ with a median absolute difference of 0.045. That difference enters τ multiplied by $f_k \approx 1$, so it is more than half the 0.08 calling threshold on its own.

4 Confidence: the p-value against what the build holds ✗

What a p-value in the build is. Two kinds of p-value sit under the same field. A record kept from one screen carries the source’s p-value unchanged: for every one of the 41,392 single-screen closure doubles and 363,818 single-screen triples, the stored p equals the source row’s p to the last digit. A record merged from k screens carries a p-value the build computed, a two-sided t -test of the k merged scores against zero with $k - 1$ degrees of freedom. Every source p-value that went into that record is gone.

The sources’ own p-values are not recomputable from anything they released. All four papers describe the null as a pooled variance: Baryshnikova 2010 combines an array-strain term from wild-type control screens with a query-strain term pooled across the screen (`si/si1.md` lines 262 to 288), and Kuzmin estimates the array term from 91 wild-type control screens (line 63). The released tables carry only the combined mutant’s own colony standard deviation. The one formula written out explicitly, for Kuzmin 2020’s query-query interactions (`si/si1.md` line 52), is a one-sided normal test of the score against the propagated variance of the expected product. What can be tested is whether that normal model, on the standard deviations the build stores, ranks the stored p-values.

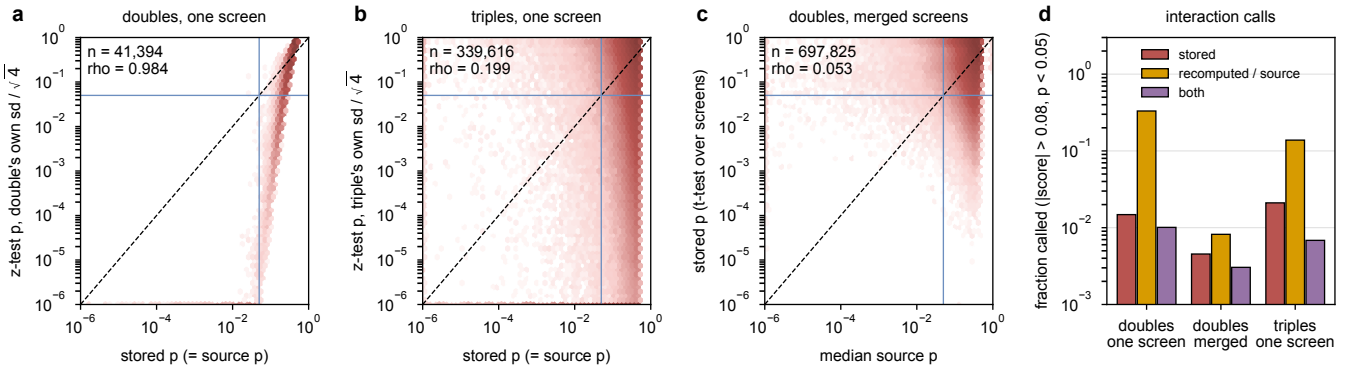


Figure 3: Stored against recomputed p-values, log-log, clipped at 10^{-6} ; blue lines at $p = 0.05$. (a) Single-screen closure doubles: the source p against a two-sided normal test of the stored score with null standard deviation $\sigma_{ab}/\sqrt{4}$, the double’s own colony SD over its four colonies. (b) Single-screen triples, the same test with the triple’s own SD. (c) Merged closure doubles: the stored t -test p against the median of the p-values the sources reported for the same pair. (d) Fraction of records called an interaction at the intermediate threshold ($|\text{score}| > 0.08, p < 0.05$) by the stored fields, by the recomputed score and p (a, b) or the source median (c), and by both; log scale.

Table 4: Stored against recomputed p-values, by stratum. ρ is Spearman’s correlation between the two p-values; the last two columns are the fraction of the stratum called an interaction at $|\text{score}| > 0.08$ and $p < 0.05$ under the stored fields and under the comparison. Generated by `s3_closure_recompute.py`.

comparison	n	ρ of p	called stored	called other
doubles, one screen: source p vs z from the double’s sd	41,394	0.984	0.015	0.036
doubles, one screen: source p vs z with the singles’ sd propagated	41,394	0.813	0.015	0.011
doubles, one screen: source call vs call on recomputed ε	41,394	0.984	0.015	0.331
doubles, merged: stored (t -test) vs z from the double’s sd	697,825	0.686	0.005	0.017
doubles, merged: stored (t -test) vs median source p	697,825	0.053	0.005	0.008
triples, one screen: source p vs z from the triple’s sd	339,616	0.199	0.021	0.110
triples, one screen: source p vs z with every term propagated	339,616	0.097	0.021	0.138
triples, merged: stored (t -test) vs source p	12,914	–	0.067	0.057

A single-screen digenic p is the double’s own colony SD. On the 41,394 single-screen closure doubles the normal test on $\sigma_{ab}/\sqrt{4}$ ranks the source p at $\rho = 0.984$ (Figure 3a), the same ordering the loader provenance found on the Kuzmin raw table. Its calls contain 606 of the source’s 613 at the intermediate threshold. The test is not the source’s test: it calls 1,505 doubles, 2.5 times the source, because the source’s pooled null is wider than one double’s four colonies. Adding the singles’ stored standard deviations to the null lowers the ranking to $\rho = 0.813$, so the stored

single-mutant SD is not the quantity the source propagated. Applying the same test to the recomputed score instead of the stored one calls 13,705 doubles, 22 times the source, with 418 in common: the recomputed score carries the join’s error but the test does not know it.

A trigenic p is reproducible from nothing in the build. On the 339,616 single-screen triples the same test on the triple’s own SD ranks the source p at $\rho = 0.199$ (Figure 3b); propagating every term of Eq. 2 through the stored SDs gives $\rho = 0.097$. The trigenic null variance is dominated by the two digenic control-screen terms, whose variance was estimated from the same-batch control screens and not released, so the triple’s colony SD carries almost none of it. The test on the stored τ recalls 0.91 of the source calls but makes 5.2 times as many; on the recomputed τ it agrees with the source on 2,323 of its 46,957 calls.

A merged p is a different statistic. On the 697,825 merged closure doubles the stored t -test p correlates with the median source p at $\rho = 0.053$ (Figure 3c), and with the normal test on the merged double’s SD at $\rho = 0.686$. The t -test measures agreement among the k duplicate scores, not the score against measurement noise, and with one or two degrees of freedom it is both too lenient and too strict: 5.4 percent of merged doubles are stored as $p < 0.05$ against 2.0 percent by the source median, while 1,754 doubles significant in every one of their sources are stored as not significant. At the intermediate threshold the stored fields call 3,169 merged doubles and the source medians 5,719, with 2,125 in common. Among the 12,914 merged triples, 505 are significant in every source and 406 of those are stored as not significant.

5 The hazards of the join, counted $\times$

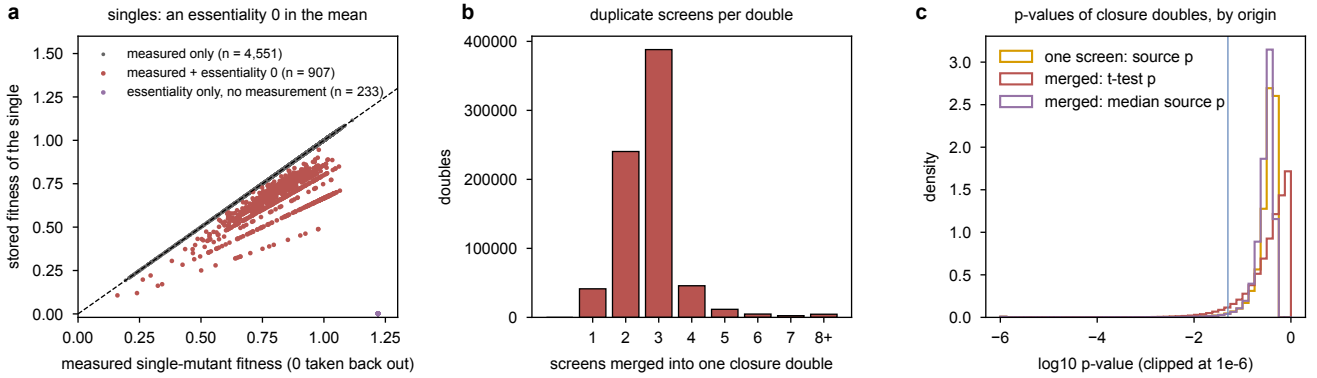


Figure 4: What the join does to the values. (a) Every single in the pool: stored fitness against the measured fitness with the essentiality 0 taken back out ($k/(k-1)$ times the stored mean for a single merged from k entries of which one is the 0). Gray, no essentiality entry, on the identity; red, 907 singles whose measured allele fitness was averaged with a 0; purple, 233 singles with an essentiality entry and no measurement, stored at 0 and placed at the right margin. (b) Number of source screens merged into each closure double. (c) Distribution of $\log_{10} p$ over closure doubles by how the p -value arose: the source’s own p for single-screen records, the build’s t -test for merged records, and the median source p for the same merged records.

Table 5: Records of the S3 pool touched by each hazard. Generated by `s3_closure_recompute.py`.

hazard	records
singles carrying an SGD essentiality entry (fitness 0 in the mean)	1,140
of which essentiality only, no measurement (stored fitness 0)	233
singles carrying a SynthLethDB entry (fitness 0)	3
records with two fitness values (masked by the trainer)	3
closure doubles touching an essentiality-tainted single	141,099
of which touching an essentiality-only single	0
triples touching an essentiality-tainted single	78,707
of which touching an essentiality-only single	0
closure doubles merged from several screens (p replaced by a t -test)	697,825
triples merged from several screens (p replaced by a t -test)	12,914
merged doubles significant in every source, not significant in the build	1,754
merged triples significant in every source, not significant in the build	406

H1: an essentiality label averaged into a measured fitness. The SGD essentiality entry is a statement about the gene, not a measurement of a strain. Costanzo and Kuzmin measured essential genes through temperature-sensitive or decreased-abundance alleles that grow, so the same gene name carries a measured allele fitness and a converted 0, and

the deduplicator’s key, experiment type plus gene set, cannot tell them apart. Of the 1,140 singles with an essentiality entry, 907 also carry measurements; their stored fitness has median 0.679 where the measured mean has median 0.843, and the four bands in Figure 4a are the values of k . Those singles enter 141,099 closure doubles and 78,707 triples, and Section 3 measured what they cost. The 233 essentiality-only singles touch no closure double or triple, so they cost the identity nothing, but they teach a fitness model that 233 genes have a fitness of exactly 0 with no colony behind it.

H2: two fitness values in one record. A SynthLethDB single-gene record lists its gene twice, so its sorted gene set differs from the Costanzo single’s and the two are not merged; the trainer then finds two fitness values in one record, 0.0 and about 1.0. Three singles do this (YPL212C, YHL047C, YKL117W), and they killed the first S3 run at epoch 0 before the decode was changed to mask such rows. The 691 closure doubles with a SynthLethDB entry merge cleanly, a measured double-mutant fitness averaged with a 0, which is H1 again at order two.

H3: cross-screen singles in a within-screen identity. Eq. 1 is exact on a source row because f_a , f_b and f_{ab} were normalized together. The build’s single is a mean over every source that measured the gene, and its double is a mean over every screen of the pair. The single-screen stratum of Table 2 isolates this: a double whose fitness and score are one screen’s, against singles that are not, reaches $r = 0.339$.

H4: the merge replaces the p-value. Section 4 measured this. The count in Table 5 is 697,825 closure doubles and 12,914 triples whose stored p-value is a t -test with one to four degrees of freedom over duplicate scores, and the 1,754 doubles and 406 triples that every source called significant and the build does not.

H5: the source substituted 1.0 for a missing fitness. The released Kuzmin 2020 table has no query fitness on 99.6 percent of its digenic rows and the reported scores were computed with 1.0 in its place; Costanzo 2016 does the same on 2.1 percent of its rows. The build stores the score and stores whatever single-mutant fitness it has for the gene from other sources, so for these pairs no fitness in the build can reproduce the score, and a model asked to learn both is asked to learn an inconsistency the source created.

6 What this means for training on S3 ✗

The identity cannot be the mechanism. The S3 arm supervises fitness and interaction jointly on the hypothesis, stated as untested in the training plan, that a representation shaped by fitness helps the interaction head. If the mechanism were the model reading Eq. 2 off its fitness predictions, its ceiling on this build would be the $r = 0.23$ that the equation itself reaches on the stored values, below the $r = 0.40$ of the additive ridge that sees no fitness at all. Whatever the S3 seeds show at readout, they are not showing that; a gain would have to come from the representation, and a null would not refute the identity, which is exact in every source.

Per-order fitness is now logged, and per-order interaction was always logged. The trainer reports fitness for singles, doubles and triples and interaction for doubles and triples separately, so the readout can ask whether the model fits the essentiality-tainted singles, which hold a value no strain has, and whether the doubles’ fitness is learned better than their interaction, which this document predicts because the double fitness is a measurement and the double interaction is a cross-screen difference.

Every p-value in a merged record is unlabeled. Nothing in the training arm reads p-values, so no run is affected. Any analysis that does, a confidence-weighted loss, a filter to significant interactions, a precision-recall against called interactions, has to treat the 697,825 merged doubles and 12,914 merged triples as carrying no p-value, and can recover the sources’ p-values only by joining back to the raw tables by gene pair, as this document did.

Changes the build would need. Three, each at the step in Figure 1 that introduces the hazard. The essentiality conversion should not produce a fitness that the deduplicator can average with a measured one: either the converted 0 is kept as its own record with a perturbation type the merge key distinguishes, or the conversion is dropped from pools that also carry measured alleles of the gene. The deduplicator should keep the source p-values beside the merged score, or combine them by a rule that reads them (Fisher’s method on the source p-values, or a normal test on the merged score with the pooled source variance), rather than a t -test on two or three values. And a merged record should carry which screens went into it, so that a within-screen identity can be evaluated within a screen; the source tables carry the screen in the strain identifier, and the build drops it at the aggregation step.

7 The same identity on a deletion-only query ✗

A second build, 029 (experiments/029-solid-growth-ko/), changes the query and the build in the way Section 6 asked for and nothing else. The query keeps only deletion alleles, `sga_kanmx_deletion` and `sga_natmx_deletion`, at

both screening temperatures, and drops the temperature-sensitive, DAmP and suppressor alleles that the 025 build merged under the same gene name; the SynthLethDB block requires two perturbation nodes, which removes the self-pairs that carried a single-gene fitness of 0. The build drops the mean-merge: every source entry of a genotype is kept beside the others, with its screen, its temperature, its marker, its standard deviation and its p-value. Figure 5 sets the two builds side by side; Table 6 counts what each holds.

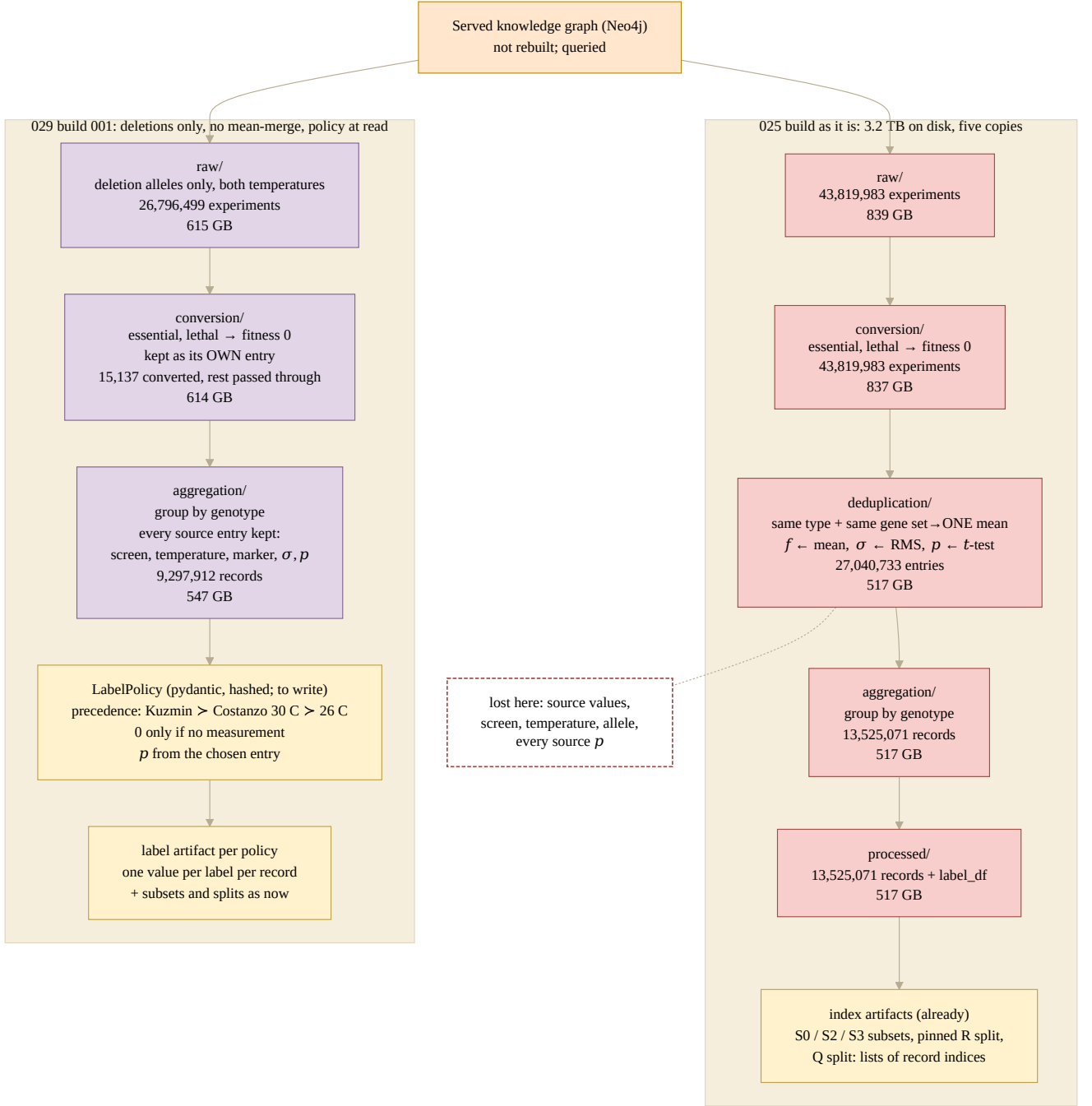


Figure 5: The two builds from the same served graph. Right: 025 as built, five full copies on disk, the mean-merge in red discarding the source values, screens, temperatures, alleles and every source p-value at the step marked. Left: 029 build 001, deletion alleles only, no merge, every source entry kept through aggregation, so that the value the trainer reads is chosen by a label policy at read time rather than fixed at build time.

A policy in place of a merge. With every entry kept, a record no longer holds one fitness. A single carries its Costanzo entries at 26 and 30 degrees under both markers, its Kuzmin query fitness where the gene was a Kuzmin query, and the converted 0 where SGD lists it as essential; a double carries both Costanzo orientations and every Kuzmin digenic screen of the pair; a triple carries one entry per Kuzmin screen. What enters the identity is a choice among them, and the recompute runs the trigenic identity of Eq. 2 under four such choices:

- *mean of every entry:* the mean over all entries, converted 0 included. This is the 025 join replayed on the

deletion-only universe.

- *measured, 0 only if none*: the mean over measured entries; the converted 0 only for a gene with no measurement at all.
- *Kuzmin first*: the Kuzmin screen of the triple’s own year, then the other Kuzmin screen, then Costanzo at 30 degrees, then 26, then the converted 0.
- *Costanzo first*: Costanzo at 30 degrees, then 26, then Kuzmin, then the converted 0.

Each policy yields one fitness per gene set, and the recomputed score is compared with every stored interaction entry; a stored entry is one source row, so the comparison can also be read per screen. The 299,146 triple gene sets carry 309,122 stored trigenic entries, because a gene set reached by both Kuzmin screens keeps both, and 289,062 of those have every term of Eq. 2 present under a policy: 15,229 of the rest are missing one of the three doubles and 8,545 one of the three singles. The comparable count in the 025 build is 352,505 of its 376,732 triples.

Table 6: Composition of the two builds and survival of the pinned splits. The 010 pinned triples are the validation and test sets of every random-split arm; a pinned triple absent from 029 was measured only with a temperature-sensitive array allele or a non-deletion query. Generated by `closure_recompute.py`.

quantity	025 build	029 build
singles	5,694	5,665
closure doubles	739,219	586,431
triples (gene sets)	376,732	299,146
triples in both builds		299,146
pinned 010 train triples present	301,386	239,663
pinned 010 validation triples present	37,673	29,842
pinned 010 test triples present	37,673	29,641
singles carrying a converted 0	1,140	1,140
of which with no measurement	233	1,116
singles with a Kuzmin query fitness	—	1,364
doubles merged across screens (p replaced)	697,825	0
triples merged across screens (p replaced)	12,914	0
interaction entries carrying the source p	single-screen only	1,848,623 of 1,848,623

What the policies reproduce. Table 8 and Figure 6 give the result. Replaying the 025 join on the deletion-only universe, the mean of every entry, gives $r = 0.435$ digenic and $r = 0.193$ trigenic, within 0.04 of the 025 build’s 0.445 and 0.230: the deletion filter alone changes little, because the loss was never in the alleles. Taking the converted 0 out of every gene that has a measurement moves both to 0.453 and 0.207. The two ordered policies then pull apart. Kuzmin first gives the best trigenic reproduction, $r = 0.245$ with slope 0.71, and the worst digenic, $r = 0.227$; Costanzo first gives the best digenic, $r = 0.508$, and the worst trigenic, $r = 0.137$. No policy is best at both orders, and the per-screen columns of Table 8 and Figure 6b say why. Each screen’s stored score is reproduced by that screen’s own fitness and by little else: the Costanzo 26 degree digenic scores at $r = 0.90$ and the 30 degree scores at 0.75 under Costanzo first, against 0.01 and 0.08 under Kuzmin first; the Kuzmin 2018 digenic scores at 0.89 under Kuzmin first, against 0.33 under Costanzo first; the Kuzmin 2018 trigenic scores at 0.52 with slope 0.93 under Kuzmin first, against 0.30. The Kuzmin 2020 rows stay at 0.41 digenic and 0.29 trigenic under every policy, which is hazard H5 of Section 5: their scores were computed with a query fitness of 1.0 that no policy over measured entries can select.

What keeps the Kuzmin-first trigenic r near 0.5 rather than near 1 is one missing measurement, and the published form of the identity can now be run directly to show that the formula’s shape is not the cause. Roles are recoverable from the build: on a triple exactly one perturbation carries an array strain identifier and the other two carry the query token, which resolves the array gene, the query pair and the query strain on all 299,146 triples. Evaluating the published asymmetric form against the symmetric one on identical records (Table 7), the 2018 screen moves from $r = 0.504$ to 0.511, a difference of 0.007. The shape is therefore excluded and the whole remaining gap on that screen is the substitute the closure is forced into for f_{ij} , the pair’s own digenic array screen in place of the query strain’s fitness, which is not a record anywhere in either build.

On the 2020 screen the shape does matter, $r = 0.278$ symmetric against 0.325 published, with the error falling from 0.212 to 0.165, and the reason is the convention of Section 3. Kuzmin 2020 releases a query fitness on 0.4 percent of its control rows and scored with 1.0 throughout. The published form needs no query single at all, so it asks the build for exactly what the source used, while the symmetric form needs all three and reconstructs values the source never had. The screen where the source substituted 1.0 is the screen where using the source’s own form pays. It is also computable on more records, 236,225 against 231,611, because it needs one double and one single where the symmetric form needs three of each.

That the identity is sensitive to its terms at all is worth checking rather than assuming. Forcing the array single fitness to 1 as a control costs 0.084 on the 2018 screen and 0.052 on 2020 and drives the 2018 slope to 1.23, so the reconstruction does respond to a wrong term, and the query double is specifically the one that is missing.

Table 7: The published asymmetric identity against the symmetric form on the 029 build, same records and the same Kuzmin-first policy with the triple’s own year promoted. The third row of each screen forces the array single fitness to 1 as a sensitivity control. Generated by `closure_recompute_asymmetric.py`.

screen	form	n	r	slope	rmse
Kuzmin 2018	asymmetric, as published	57,451	0.511	0.92	0.079
	symmetric	57,451	0.504	0.91	0.079
	array single forced to 1	57,451	0.427	1.23	0.146
Kuzmin 2020	asymmetric, as published	236,225	0.325	0.93	0.165
	symmetric	231,611	0.278	0.90	0.212
	array single forced to 1	236,442	0.273	0.89	0.212

The converted 0 changes character on this query. In 025, 907 of the 1,140 essential genes carried a measured fitness beside the 0, because the temperature-sensitive alleles were their measurements; in 029, 1,116 of them carry nothing else, so the 0 is the only value and every double or triple that would touch one is absent from a deletion screen by construction. The 0 no longer taints a measured mean; it is a label for a gene with no record. And with no merge, every one of the 1,848,623 stored interaction entries carries its source p-value, so the merged-record hazard H4 does not exist in this build: the stored p-value orders the normal test on the double’s own colony standard deviation at $\rho = 0.87$ over every digenic entry, and at 0.20 over the trigenic entries, whose source null pools variances that were never released.

Table 8: The identity under each label policy. The stored score is the source’s; the recomputed score uses the policy’s fitness for every term. Bold: the highest r per order. Generated by `closure_recompute.py`.

build, policy	stored score	n	r	ρ	slope	rmse
<i>digenic</i>						
025, as built	merged record	728,856	0.445	0.493	0.99	0.072
029, mean of every entry (the 025 join)	every stored entry	1,526,362	0.435	0.399	0.48	0.050
029, measured entries; 0 only without a measurement	every stored entry	1,526,362	0.453	0.406	0.47	0.047
029, Kuzmin first	every stored entry	1,526,362	0.227	0.194	0.46	0.095
029, Costanzo first	every stored entry	1,526,362	0.508	0.481	0.48	0.042
<i>trigenic</i>						
025, as built	merged record	352,505	0.230	0.224	0.63	0.171
029, mean of every entry (the 025 join)	every stored entry	289,062	0.193	0.201	0.54	0.172
by screen	Kuzmin 2018	57,451	0.362	0.282	0.71	0.101
by screen	Kuzmin 2020	231,611	0.182	0.224	0.56	0.186
029, measured entries; 0 only without a measurement	every stored entry	289,062	0.207	0.224	0.50	0.152
by screen	Kuzmin 2018	57,451	0.441	0.299	0.78	0.087
by screen	Kuzmin 2020	231,611	0.220	0.278	0.58	0.164
029, Kuzmin first	every stored entry	289,062	0.245	0.253	0.71	0.187
by screen	Kuzmin 2018	57,451	0.517	0.441	0.93	0.080
by screen	Kuzmin 2020	231,611	0.287	0.337	0.91	0.206
029, Costanzo first	every stored entry	289,062	0.137	0.120	0.34	0.160
by screen	Kuzmin 2018	57,451	0.302	0.180	0.66	0.111
by screen	Kuzmin 2020	231,611	0.127	0.132	0.35	0.170

Where the bias sits. The identity does not fail in the join; it fails wherever a score from one screen is tested against fitness from another. Costanzo’s and Kuzmin’s scores are each exact against their own controls and each about as far from the other’s as the 025 build’s merged values were from both, so a build that averages the screens reproduces neither, and a build that ranks them reproduces one. Which one is a decision the query and the policy make on the modeler’s behalf, and Figure 6b is the cost of each choice stated per screen. For the trigenic target the Kuzmin-first policy is the consistent one: the triples are Kuzmin’s, their digenic terms are Kuzmin’s, and the singles are Kuzmin’s where Kuzmin measured them, which is what the source did. For a digenic target the same policy discards Costanzo’s standard for the 902 thousand Costanzo digenic entries in the closure, at $r = 0.08$. A model trained on both orders under one policy is asked to hold two standards for the same gene at once, and no fitness representation satisfies both; the per-order metrics of the S3 arm are where that shows first.

The pinned splits transfer at 79 percent. Of the 37,673 validation and 37,673 test triples of the 010 random split, 29,842 and 29,641 exist in 029; the rest were measured only with a temperature-sensitive array allele or a non-deletion query. A 029 arm evaluates on the surviving triples, and its comparison with the 025 arms is restricted to the same 59,483 triples on both sides.

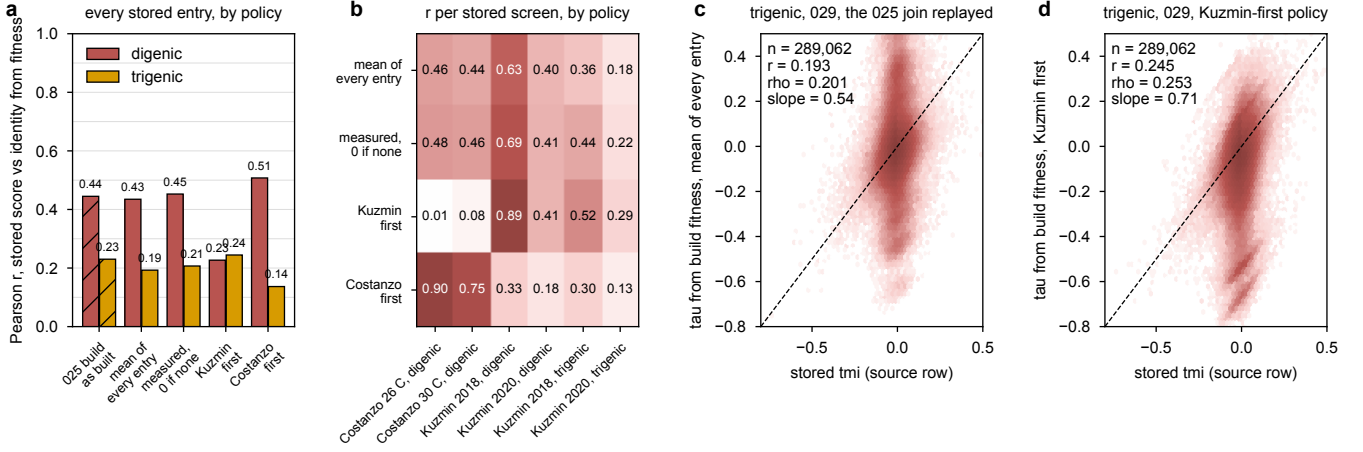


Figure 6: The identity under each label policy. (a) Pearson r between stored and recomputed score over every stored entry, digenic in red and trigenic in orange, for the 025 build as built (hatched) and for the 029 build under the four policies. (b) The same r per stored screen: each row a policy, each column the screen whose stored scores are compared. (c) Stored trigenic score against the score recomputed under the mean of every entry, the 025 join replayed on 029. (d) The same under the Kuzmin-first policy.

8 The multi-measurement build $\times$

The missing record of Section 7 is now a record. Build 030 (`experiments/030-solid-growth-multi/`, build 001, 2026-09-24) runs the 025 query, every allele, on the 029 store shape, every source entry kept, and both Kuzmin double-mutant fitness loaders emit the query strain’s own fitness from the released query strain tables as of commit 4c4a4f950: 172 Kuzmin 2018 and 200 Kuzmin 2020 double-mutant query strains, each carrying the strain identifier `GENE1+GENE2_tmNNNN` that the triples of that screen carry on their query perturbations. Because each query strain was screened against hundreds of array genes, those 372 records cover 86,111 of the 91,111 Kuzmin 2018 triples and 255,296 of the 300,187 Kuzmin 2020 triples. The store holds 43,820,352 raw records in 13,525,088 genotype groups; roles resolve on all 376,732 triples, and every perturbation carries its strain identifier, so a triple’s array strain is readable beside its query strain.

The same identity, term by term from the build. Table 9 scores the published form on identical records under the same Kuzmin-first policy with the triple’s own year promoted, with the terms chosen four ways. The 029 reading takes f_{ij} from the pair’s digenic array screen with the query-strain entries excluded; it reproduces 029 to three decimals on both screens on the deletion-array stratum, 0.511 and 0.325, so the extra alleles of the 025 query and the larger store change nothing on that reading and the two builds are comparable term by term. The second reading takes f_{ij} by `LabelPolicy.select_double`, which prefers the entry whose strain token equals the triple’s query strain. The third also matches the control terms on the triple’s own array strain, which is how the source computed them (Section 3): ϵ_{ik} and ϵ_{jk} from the single-mutant control queries’ scores against that array strain in the triple’s own screen, and f_k from that strain’s own single, the screen’s array single where the build holds one and Costanzo’s single of the same deletion strain otherwise. The fourth is a diagnostic, not a build value: f_k taken from the array single-mutant fitness the source wrote on the trigenic row itself, read from the raw tables.

Kuzmin 2018 closes. On the 86,111 triples whose query double is in the build, the query-strain double alone moves r from 0.435 to 0.951 and the median absolute residual from 0.046 to 3.3×10^{-5} ; matching the control terms on the array strain takes it to $r = 0.987$, rmse 0.008, median residual 2.9×10^{-5} , against the 0.985 of the raw-table recompute. The residual is the rounding of the published five-decimal score: the build reproduces the source. The raw-table f_k changes nothing on this screen, because the 2018 array single the build holds (`SmfKuzmin2018Dataset`, from the released array single-mutant fitness) is the row value. Over all 91,111 triples, where the 5,000 without a query double fall back to the array screen, r is 0.747; on the deletion-array stratum of 029, 0.803 against 029’s 0.511.

Kuzmin 2020 is two screens, and the build does not know which is which. The query-strain double moves the 2020 screen from 0.338 to 0.660 on its 255,064 covered triples, and matching the control terms on the array strain changes nothing (0.660, median residual 0.031), although the interactions matched on 300,185 of 300,187 triples. What did not match is f_k : the build holds no Kuzmin 2020 array single-mutant fitness, so all 299,862 values came from Costanzo’s single of the same strain. The 2020 loader emits singles from the query strain standard (Table S5) only; the array single-mutant fitness of that screen is a column on the interaction rows of Tables S1 and S3 that no loader reads. With the row’s own value in its place, everything else still from the build, r is 0.920, rmse 0.025 and the median residual 2.8×10^{-5} .

The rest is the two tables. Kuzmin 2020 released two screens: Table S1, the 240 double-mutant queries and their 480 single-mutant controls against the diagnostic array of about 1,200 strains, and Table S3, pilot screens of 11 double-mutant queries against the genome-wide deletion array and the temperature-sensitive array. The supplementary text says the array single-mutant fitness came from the 2018 study, but the value on the rows is a constant of the array strain inside each table and differs between the tables for 1,159 of the 4,553 strains, median range 0, 90th percentile 0.046, maximum 0.41; it is not the 2018 value either, agreeing with it on 1 percent of the shared strains at $r = 0.87$ for S1. The loaders read both tables into one dataset and record nothing of which table a row came from, so a 2020 triple, its controls and its array single can each come from either screen. Recomputed on the raw tables with every term taken inside the row’s own table (`kuzmin2020_within_table_recompute.py`), the identity reproduces the released score at $r = 1.000$, rmse 3.6×10^{-5} , every residual below 10^{-3} , on all 256,804 trigenic rows with both controls; pooling the controls over the two tables gives 0.977, the within-screen figure of Section 3, and averaging f_k over them 0.906. The 2020 score is therefore exactly recoverable from released data, and what the build needs is two things the loaders drop: the array single-mutant fitness per array strain and table, and the table itself on every 2020 record, as the screen identity a policy matches on.

Table 9: The published identity on the 025, 029 and 030 builds. 025 is the merged build under the symmetric form (Section 3); 029 and 030 use the Kuzmin-first policy with the triple’s own year promoted. “array-screen double” is the 029 reading; “query-strain double” takes f_{ij} from the double-mutant query strain’s own record; “array-strain controls” also matches ϵ_{ik} , ϵ_{jk} and f_k on the triple’s array strain; the last form is the diagnostic with f_k from the source’s own row. “Deletion” is the stratum whose three genes carry deletion alleles, the 029 universe; “every term strain-matched” is the stratum whose query strain has a record and whose control terms matched. Generated by `closure_recompute_030.py`.

build	screen, stratum	form	n	r	slope	rmse
025	both	symmetric, mean over entries	352,505	0.230	0.63	0.171
029	Kuzmin 2018 deletion	symmetric	57,451	0.504	0.91	0.079
		array-screen double (029 reading)	57,451	0.511	0.92	0.079
029	Kuzmin 2020 deletion	symmetric	231,611	0.278	0.90	0.212
		array-screen double (029 reading)	236,225	0.325	0.93	0.165
030	Kuzmin 2018 deletion	array-screen double (029 reading)	57,445	0.511	0.92	0.079
		query-strain double	57,877	0.778	0.87	0.037
		+ array-strain controls	57,877	0.803	0.88	0.034
		+ f_k from the source row (diagnostic)	57,877	0.803	0.88	0.034
030	Kuzmin 2018 every term strain-matched	array-screen double (029 reading)	85,206	0.435	0.96	0.102
		query-strain double	86,111	0.951	0.98	0.016
		+ array-strain controls	86,111	0.987	0.99	0.008
		+ f_k from the source row (diagnostic)	86,111	0.987	0.99	0.008
030	Kuzmin 2020 deletion	array-screen double (029 reading)	236,264	0.325	0.93	0.165
		query-strain double	250,062	0.543	0.94	0.090
		+ array-strain controls	250,023	0.548	0.94	0.089
		+ f_k from the source row (diagnostic)	250,293	0.651	0.96	0.068
030	Kuzmin 2020 every term strain-matched	array-screen double (029 reading)	238,537	0.338	0.91	0.160
		query-strain double	254,977	0.659	0.92	0.068
		+ array-strain controls	254,977	0.660	0.93	0.068
		+ f_k from the source row (diagnostic)	254,977	0.920	0.94	0.025

9 Limits ✗

The recomputed p-values are not the sources’ p-values. Section 4 is a ranking comparison. The normal test on the double’s own colony SD orders the source p-values at $\rho = 0.984$ but calls 2.5 times as many interactions, because the source null pools an array-strain and a query-strain variance from control screens that were never released. No recomputation from the build or from the released tables can recover those; the statement is that the source p is reproducible in rank, not in value.

The measured-mean restoration assumes exactly one 0 in the mean. The $k/(k-1)$ correction in Section 5 assumes a tainted single merged exactly one essentiality entry of value 0 with $k-1$ measured entries. The `num_duplicates` field gives k and the source list confirms one essentiality entry for every tainted single, but the individual measured values are gone after the merge, so the restored mean is exact only if the converter wrote exactly 0. It did (`CompositeFitnessConverter`), and the two checked cases (YER048W-A, stored 0.452 from 0.904 and 0; YDR292C, stored 0.630 over $k = 7$) agree.

The within-screen trigenic control matches control queries by array strain, not by batch. Section 3

matches a trigenic row to its two single-mutant control query rows on the array strain identifier and the control gene. The supplementary methods say those controls were screened in the same batch as the double-mutant query, but the released tables carry no batch field, only the query strain identifier (`tm` number), which differs between a double-mutant query and its controls. A gene was screened against the same array strain more than once in 2.8 percent of the 2018 matches and 7.6 percent of the 2020 matches, and those are averaged. The residual this leaves is small, $r = 0.985$ and a median absolute residual of 0.0016, so the matching is not what limits that control.

The trigenic score reproduces itself at 0.59, and two published figures differ. Two numbers are in circulation for the replicate reproducibility of the adjusted trigenic score, both from the Kuzmin 2018 screens. The supplementary text reports correlations between independent replicates of 0.90 to 0.91 for double-mutant and raw triple-mutant scores and 0.74 to 0.81 for the adjusted trigenic score (`si/si1.md` line 171), but the panel those come from plots only interactions called significant at $p < 0.05$. That p-value is a normal test of the score against a variance propagated from the query and array fitness estimates, so conditioning on it keeps the scores that are large relative to their own measurement error and drops the rest, which raises a correlation computed within the subset. Over the whole distribution the number is lower: Dango (`zhangDANGOPredictingHigherorder2020`) recomputed it on all 91,050 triples of the Kuzmin 2018 diagnostic-array screen and reports “The Pearson correlation between the trigenic interaction scores of two individual replicates is around 0.59, which is much lower than the Pearson correlation between the digenic interaction score of two replicates from the same data source (0.88)”.

The 0.59 is the honest figure for an unselected pool such as S3, and it is the right one to read the recompute against, with two qualifications. It is the correlation between two single replicates, so it is what a replicate oracle achieves rather than a mathematical ceiling; a model predicting the denoised value and scored against a noisy one is limited instead by the square root of the target’s reliability, which for a score combining two replicates of individual reliability 0.59 is about 0.86. And it is measured on one screen’s diagnostic array, not on the pooled S3 or 029 triples. With those held in mind, the Kuzmin-first policy’s $r = 0.517$ on the 2018 screen (Section 7) sits just under the value two repetitions of the experiment attain against each other, so the identity recomputed from the database is close to as consistent with the released score as the measurement is with itself.

Four colonies is a documented representative, not a per-record count. The null standard deviation $\sigma_{ab}/\sqrt{4}$ uses the colony count the loaders record (Costanzo 2016: “4 replicate colonies per double mutant”; the Kuzmin loader back-solved $n = 4$ against the reported p-values). Only 41,370 of the closure doubles carry an explicit `n_samples`; the rest use the same value. A different n rescales every recomputed p monotonically and changes no ranking in Table 4.

Nothing here reads a training run. Every number is a property of a pool. The three S3 seeds (IGB `mm1i` 2409033, 2409031, 2409032) were running or queued when this was written and are read out elsewhere.